

Visual-Fusion-GRU

Final Project Report

SLA-Aware Underwater VLA Navigation with RL, CLIP Heatmaps, GRU Imitation, and
Hard-Case DAgger

Quanlong Zhao

Student ID: 999014947

Department Project Submission

June 2026

Headline result: the current best GRU imitation student reaches **0.957 success rate**
over 1000 pure-CLIP evaluation episodes after hard-case DAgger fine-tuning.

Contents

1	Abstract	3
2	Repository and Project Scope	3
2.1	Main Repository Components	3
3	Problem Definition	4
4	Original RL Plan	4
5	RL Choke Point	6
6	Shift to Imitation Learning	7
6.1	Behavior Cloning Baseline	7
6.2	Dagger	8
7	GRU Student Model	8
8	Training Data	9
9	Evaluation Results	10
9.1	Residual Hard Cases	11
10	Why the Final Approach Worked	11
10.1	RL Found the Bottleneck	11
10.2	Dagger Matched the Student’s Actual State Distribution	12
10.3	GRU Memory Reduced Target-Loss Oscillation	12
10.4	Hard-Case Mining Focused the Remaining Learning	12
11	Reproducibility	12
12	Limitations	13
13	Future Work	13
14	Conclusion	13
A	Key Artifact Paths	14

1 Abstract

This report describes the design, training history, and final experimental outcome of Visual-Fusion-GRU, a GitHub-ready underwater navigation project for an autonomous robotic fish in NVIDIA Isaac Sim. The project goal is a Vision-Language-Action (VLA) style policy: given an instruction such as **Find the red ball**, the robot should ground the instruction in camera observations and choose low-level swimming actions. The work began with reinforcement learning (RL) and an alpha-fusion curriculum that attempted to migrate from a stable color-saliency fallback heatmap to a CLIP semantic heatmap. The RL experiment revealed a repeatable bottleneck near **alpha = 0.993**, where only 0.007 fallback weight remained, but success dropped sharply. This showed that even a very small fallback component still carried high-quality control decisions.

The final successful path used imitation learning instead of direct RL annealing. A fallback RL policy was preserved as the teacher. A CLIP-observation student was trained with behavior cloning (BC), then DAgger, then a recurrent GRU policy, and finally hard-case DAgger fine-tuning. The current best released checkpoint is `models/student_gru_hard_cases_radius1_seed601000.pt`. It achieves **0.957 success rate over 1000 pure-CLIP episodes**, with mean steps **31.341**. The final fine-tuning stage used **73737** labeled records from the balanced teacher seed dataset, eight balanced GRU DAgger shards, and a radius-1 hard-case dataset.

2 Repository and Project Scope

The GitHub package is stored in:

`/home/quanlong/isaac-sim/Visual_Fusion_GRU_github`

The repository is curated from a larger Isaac Sim workspace. It contains source code, model checkpoints, configs, evaluation summaries, monitor logs, DAgger datasets, and upload notes. The full Isaac Sim installation and local TensorBoard event folders are not intended to be the core public artifact.

2.1 Main Repository Components

Path	Purpose
<code>src/underwater_env.py</code>	Isaac Sim underwater navigation environment, including observation construction and low-level action stepping.
<code>src/vla_clip.py</code>	CLIP-based semantic heatmap construction for the language-conditioned observation path.
<code>src/train_phaseb_hybrid.py</code>	Historical hybrid RL training entry point for the alpha-fusion curriculum.
<code>src/train_fallback_teacher.py</code>	Fallback teacher RL training entry point.
<code>src/imitation/</code>	Behavior cloning, DAgger, datasets, configs, policy adapters, evaluation helpers, and model definitions.

Path	Purpose
<code>src/imitation/models.py</code>	MLP and GRU student policies for heatmap-to-action classification.
<code>configs/imitation/</code>	JSON configs for BC, DAgger, GRU DAgger, evaluation, teacher dataset collection, and fallback teacher training.
<code>models/</code>	Compact trained artifacts, including fallback teacher, historical RL checkpoints, BC baselines, and the current best GRU student.
<code>imitation_data/</code>	Teacher seed datasets and evaluation summaries.
<code>imitation_runs/</code>	DAgger datasets, checkpoints, and per-run summaries.
<code>monitor_logs/</code>	Monitor CSV logs for fallback and hybrid RL runs.
<code>docs/</code>	GitHub upload notes, project structure, next-experiment notes, and assets.

3 Problem Definition

The robotic fish must navigate underwater toward a target object while using a vision-language style observation. The target instruction used throughout the recorded experiments is:

Find the red ball

The action space is discrete:

- **0**: swim forward,
- **1**: turn left,
- **2**: turn right.

The semantic observation used by the imitation student is a flattened CLIP heatmap with input dimension **196**, corresponding to a 14×14 spatial heatmap. This compact representation is used by both the MLP and GRU student models.

The environment is challenging because underwater visual navigation does not only require moving toward a visible object. The policy must also avoid left-right oscillation when the target leaves the camera view, recover from target loss, and decide when to keep moving forward instead of over-correcting with turns.

4 Original RL Plan

The original project hypothesis was that a stable color-saliency fallback policy could teach the physical target-tracking behavior, and then an RL curriculum could gradually replace fallback geometry with semantic CLIP heatmaps. The hybrid observation was:

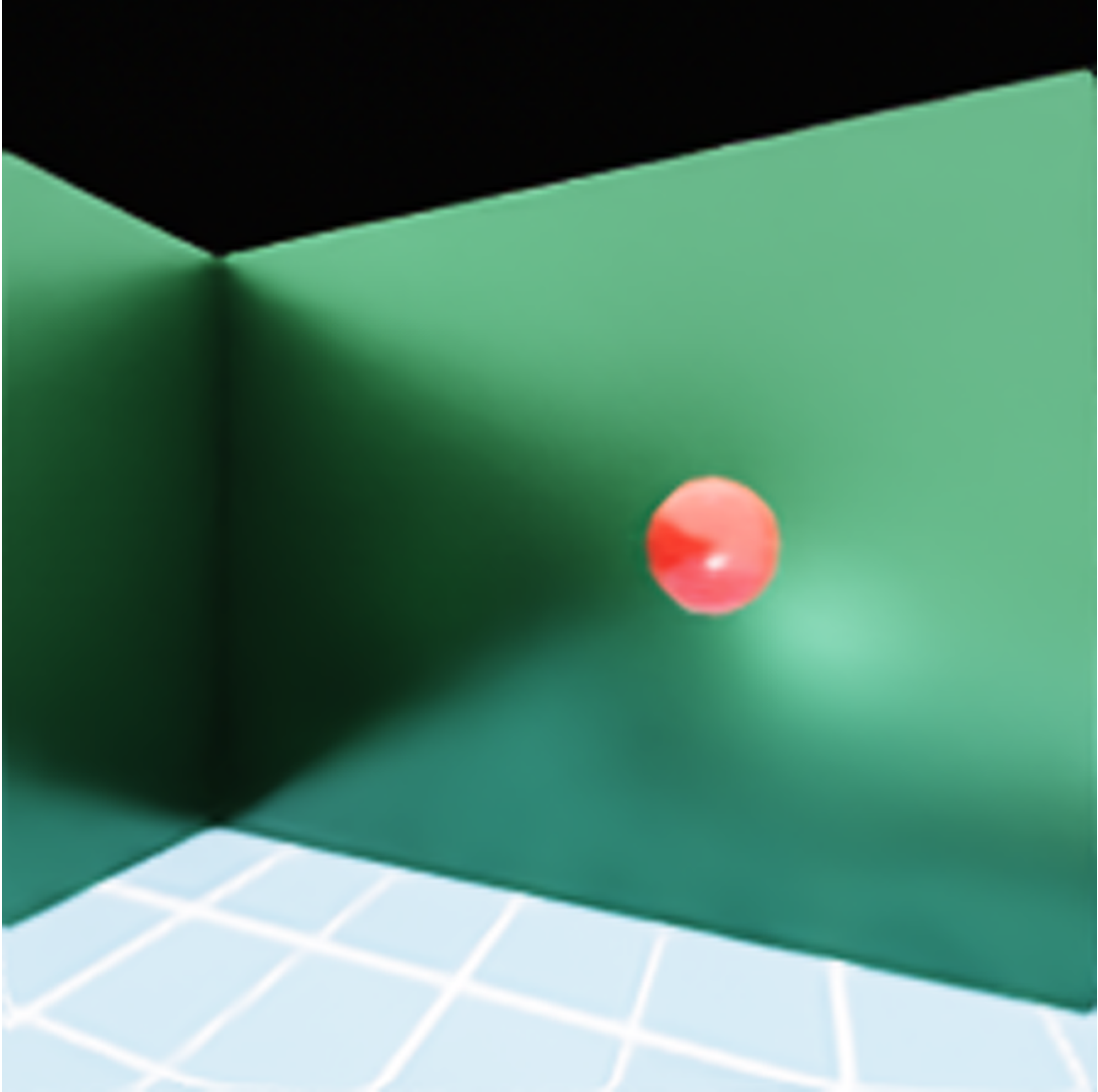


Figure 1: Current GRU student simulation screenshot included with the GitHub package.

$$obs = \alpha \cdot CLIP_heatmap + (1 - \alpha) \cdot fallback_heatmap.$$

The intended roadmap was:

1. train a fallback RL policy on color saliency;
2. use the fallback policy as a stable target-tracking base;
3. introduce CLIP semantic heatmaps with alpha fusion;
4. slowly increase α until $\alpha = 1.0$;
5. remove the fallback path and keep a pure semantic policy.

This was a reasonable first plan. The fallback signal was dense and physically meaningful: when the red target was visible, its heatmap peak gave a clean left-center-right steering cue. CLIP was more semantically aligned with language, but less reliable as a metric control signal. The alpha curriculum was intended to bridge that representation gap.

5 RL Choke Point

The Phase B hybrid RL experiment tested the migration directly. The key run is documented by:

- config: `models/run_config_phaseb_hybrid.json`,
- monitor log: `monitor_logs/monitor_phaseb_resume_992_softgate_20260420_130251.monitor.csv`,
- launch script: `scripts/run_phaseb_resume_990to1000.sh`.

The curriculum moved from $\alpha = 0.992$ toward $\alpha = 1.0$, using small 0.0005 alpha milestones and a 0.9 success-rate gate. The run stalled before reaching pure CLIP:

Table 2: Hybrid RL alpha-curriculum bottleneck.

Alpha	Episodes	Successes	Success rate
0.9920	80	73	0.912
0.9925	128	114	0.891
0.9930	2368	1602	0.677
0.9935	1244	643	0.517

This was the most important negative result in the project. At $\alpha = 0.993$, the fallback contribution is only $1 - \alpha = 0.007$, or 0.7% of the fused heatmap. Yet removing this tiny part caused a major drop in success. The interpretation is that the fallback signal was small in percentage but large in

decision quality. It still supplied critical low-level steering information in states where the CLIP heatmap was too noisy or too semantically broad to support stable control.

The observed failure mode was not just a bad hyperparameter setting. It was a representation mismatch. The fallback heatmap represented target geometry in a control-friendly way, while the CLIP heatmap represented semantic likelihood. The policy needed a stable steering gradient, especially after target loss. As α approached 1.0, PPO kept collecting on-policy data from increasingly unstable behavior, including search loops and left-right shaking. This made direct RL annealing a poor route to the final pure-CLIP objective.

6 Shift to Imitation Learning

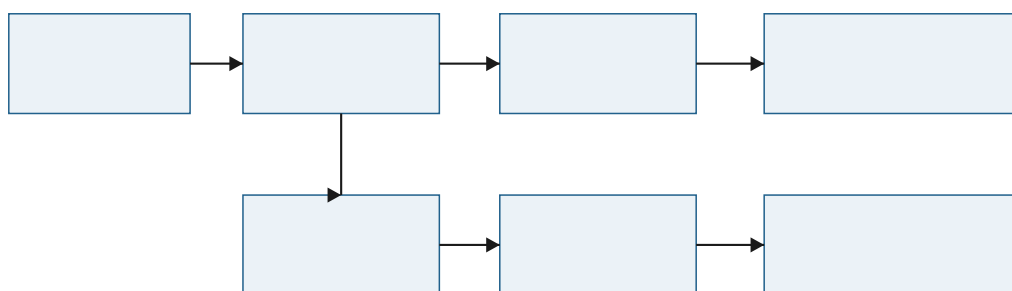
The project did not discard the fallback teacher. Instead, the successful decision was to keep the useful part of the RL work and change the transfer method. The fallback policy became a teacher, while the student observed only the CLIP heatmap. The central question changed from:

Can PPO discover how to migrate from fallback to CLIP?

to:

Can a CLIP-view student imitate the decisions of a fallback teacher, including off-trajectory recovery states?

Training Pipeline Used in Visual-Fusion-GRU



The deployed student uses pure CLIP observations; fallback is retained as teacher and diagnostic signal.

Figure 2: Training pipeline: RL exposed the semantic-control bottleneck, then imitation learning used the fallback policy as teacher and trained a pure-CLIP GRU student.

6.1 Behavior Cloning Baseline

The first imitation baseline used behavior cloning on teacher-labeled examples. The balanced teacher seed dataset contains **360** teacher episodes and **11720** labeled records:

`imitation_data/teacher_seed_balanced/teacher_seed_balanced_dataset.summary.json`

The early BC-only CLIP student was weak. In the recorded balanced evaluation, `models/student_bc_balanced.pt` achieved only **0.15** success rate over **20** episodes. Its action histogram was:

`{"0": 108, "1": 4904, "2": 4912}`

The almost symmetric left/right counts indicate the main problem: the model often oscillated instead of committing to a useful tracking or search action. This made DAgger necessary.

6.2 DAgger

DAgger corrects a limitation of pure BC. BC trains on teacher states, but at test time the student visits states created by its own mistakes. DAgger collects the states the student actually visits, asks the teacher for labels in those states, adds the corrected examples to the dataset, and trains again.

The balanced GRU DAgger configuration is stored at:

`configs/imitation/dagger_gru_balanced.json`

Key settings were:

- **model type:** GRU,
- **total iterations:** 8,
- **rollout episodes per iteration:** 40,
- **scenario plan:** balanced,
- **in-view runs per iteration:** 20,
- **out-of-view runs per iteration:** 20,
- **beta schedule:** 0.8 to 0.2,
- **evaluation episodes per iteration:** 25.

The balanced scenario plan was important. It forced DAgger to collect both normal visible-target tracking states and harder target-loss/search states.

7 GRU Student Model

The recurrent student is implemented in:

`src/imitation/models.py`

The final student uses `HeatmapActionGRU`. It consumes the flattened CLIP heatmap at each timestep and keeps a recurrent hidden state during rollout. This gives the model short-term memory of where the target was before it disappeared from the current camera view. The model predicts one of the three discrete actions at each step.

The main architecture configuration is:

Table 3: GRU student architecture and training configuration.

Parameter	Value
Input dimension	196
GRU hidden dimension	128
GRU layers	1
MLP head hidden dimensions	[128]
Activation	ReLU
Dropout	0.0
Batch size	128
Initial epochs	20
Initial learning rate	0.001
Weight decay	0.0001
Sequence length	16
Sequence stride	4
Device in config	CPU

Compared with an MLP, the GRU directly targets the project’s hardest control case: target loss. A single heatmap can be ambiguous when the target is partly or fully out of frame. A short history lets the student remember whether it was last turning left, turning right, or aligned enough to move forward.

8 Training Data

The final best model was not produced by one large undifferentiated training run. It was built through a sequence of increasingly targeted datasets:

Table 4: Dataset ladder used for the best GRU student.

Dataset stage	Episodes	Records	Role
Balanced teacher seed	360	11720	Initial fallback teacher labels
Balanced GRU DAgger shards	320	26444	Off-policy student recovery data
First hard-case shard	119	4905	Failed-seed correction after broad GRU eval
Radius-1 hard-case shard	299	35573	Expanded hard cases around seed family 601000

The final radius-1 fine-tune used **73737** total labeled records across ten dataset paths. The training summary is:

```
imitation_runs/dagger_gru_hard_cases_radius1_seed601000/summaries/hard_seed_
dataset_train_summary.json
```

This final stage resumed from the previous hard-case student:

```
imitation_runs/dagger_gru_hard_cases_1000/checkpoints/student_hard_seed.pt
```

It then fine-tuned for **2** additional epochs at learning rate **5e-5**, producing:

```
imitation_runs/dagger_gru_hard_cases_radius1_seed601000/checkpoints/student_hard_
seed.pt
```

A GitHub-friendly copy of this checkpoint is stored as:

```
models/student_gru_hard_cases_radius1_seed601000.pt
```

9 Evaluation Results

Evaluation Success-Rate Ladder

Pure CLIP student evaluations unless noted otherwise

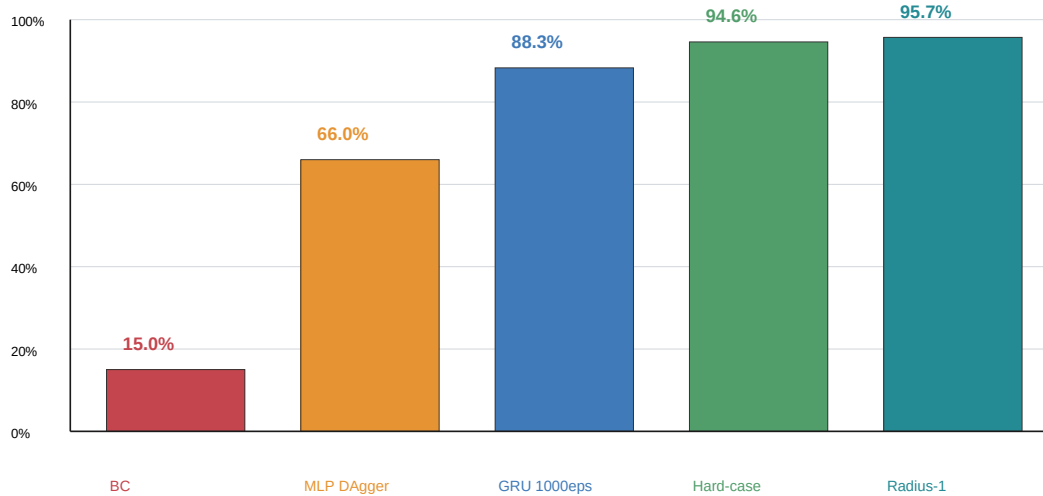


Figure 3: Success-rate ladder from BC through final radius-1 hard-case GRU fine-tuning.

The evaluation ladder shows the effect of each major design change:

The final policy’s action histogram over the 1000-episode evaluation was:

```
{"0": 17409, "1": 2404, "2": 11528}
```

Table 5: Main policy evaluation results.

Policy	Episodes	Success rate	Mean reward	Mean steps
BC-only MLP baseline	20	0.150	54.156	496.200
MLP DAgger iter04	100	0.660	34.682	46.130
GRU DAgger iter08	100	0.880	51.609	32.010
GRU DAgger iter08	200	0.920	53.484	29.370
GRU DAgger iter08	1000	0.883	51.587	31.346
Hard-case GRU	1000	0.942	60.931	79.066
Hard-case GRU, seed 601000 family	1000	0.946	60.857	69.151
Radius-1 hard-case GRU	1000	0.957	56.392	31.341

The best result is supported by:

- `imitation_data/eval/dagger_gru_hard_cases_radius1_seed601000_1000eps/student_hard_seed_eval_1000eps.json`,
- `imitation_data/eval/dagger_gru_hard_cases_radius1_seed601000_1000eps/student_hard_seed_hard_cases.json`,
- `models/CURRENT_BEST_STUDENT.md`.

9.1 Residual Hard Cases

The latest hard-case report contains **48** hard cases over 1000 episodes:

- 43 outright failures,
- 0 near misses under the configured near-miss rule,
- 5 slow successes.

This is a useful residual set. The broad behavior has mostly been learned, so the next improvements should focus on targeted recovery rather than simply adding more random episodes. However, because hard-case tuning can overfit, any new fine-tune should also be evaluated on a wider unseen seed range.

10 Why the Final Approach Worked

The final result came from combining three insights:

10.1 RL Found the Bottleneck

The alpha-fusion RL experiment did not reach $\alpha = 1.0$, but it produced a valuable diagnosis. The failure near $\alpha = 0.993$ showed that the last small fallback component was not redundant. It was

doing important work in the control loop. This prevented the project from spending more time trying to force PPO through an unstable representation transition.

10.2 Dagger Matched the Student’s Actual State Distribution

BC trained on teacher states, but the student made mistakes and entered states not well represented in the initial teacher dataset. Dagger fixed this by collecting labels for states the student actually visited. The MLP Dagger baseline already improved strongly over BC, and the GRU version improved the target-loss behavior further.

10.3 GRU Memory Reduced Target-Loss Oscillation

The target can temporarily leave the camera field of view. A single CLIP heatmap at that moment can be weak or ambiguous. The GRU hidden state gives the student temporal context: it can remember recent target direction and avoid resetting to symmetric left-right shaking. This directly addresses the failure mode observed in early BC and RL rollouts.

10.4 Hard-Case Mining Focused the Remaining Learning

After the broad GRU policy reached 0.883 success over 1000 episodes, its failed seeds were extracted and used as targeted teacher-labeled data. The first hard-case fine-tune raised performance into the 0.94 range. Expanding the hard-case seed set with radius 1 produced 299 targeted episodes and pushed the current best to 0.957.

11 Reproducibility

The release package contains helper scripts for the current workflow:

Command	Purpose
<code>./scripts/eval_student_release.sh</code>	Evaluate the public current-best checkpoint using <code>configs/imitation/eval_current_best_student.json</code> .
<code>./scripts/run_dagger_gru_release.sh</code>	Run the balanced GRU Dagger release configuration.
<code>./scripts/eval_dagger_gru_students_200eps.sh</code>	Evaluate GRU Dagger checkpoints over 200 episodes.
<code>./scripts/eval_iter08_1000eps_and_extract_hard_cases.sh</code>	Evaluate <code>student_iter_08.pt</code> over 1000 episodes and extract hard cases.
<code>./scripts/run_gru_hard_seed_dagger.sh</code>	Collect failed-seed labels and fine-tune a GRU student.

The code expects NVIDIA Isaac Sim / Omniverse Python APIs to be available. The repository’s `requirements.txt` lists regular Python packages only; it does not install Isaac Sim itself.

12 Limitations

The project has a strong current result, but it is not fully solved.

- The best model still has 43 failures and 5 slow successes in the latest 1000-episode evaluation.
- The language instruction is fixed to **Find the red ball**; broad natural-language generalization remains future work.
- The final hard-case fine-tune is targeted around a seed family. It should be validated on wider unseen seed ranges before being treated as a final robust policy.
- CLIP heatmaps are semantic but not always metric-control-friendly. The fallback teacher remains valuable as a diagnostic signal even when the deployed student uses pure CLIP observations.
- Full Isaac Sim runtime assets are larger than the curated GitHub package and must be maintained locally.

13 Future Work

The next steps should be conservative and evidence-driven:

1. Re-evaluate the current best checkpoint on wider seed ranges beyond the 601000 family.
2. Mine the remaining 48 hard cases and collect a small teacher-labeled shard.
3. Fine-tune for only 1 to 2 epochs at a low learning rate, then check both the hard-case family and the broad benchmark.
4. Add richer target-loss scenarios where the target disappears for longer windows.
5. Extend the semantic setup beyond a single red-ball instruction once the current control behavior is stable.

14 Conclusion

Visual-Fusion-GRU demonstrates a complete research arc: an RL idea was tested, failed in a measurable way, and then became the diagnostic foundation for a better imitation-learning solution. The fallback RL teacher proved that the robotic fish could learn physical tracking behavior. The alpha-fusion curriculum proved that the last 0.7% fallback signal could still carry critical control decisions. The final GRU imitation pipeline used that insight to train a pure-CLIP student with temporal memory and hard-case DAGger.

The final released model, `models/student_gru_hard_cases_radius1_seed601000.pt`, reaches **95.7%** success over **1000** pure-CLIP evaluation episodes. This is the current best result in the GitHub package and the recommended baseline for future experiments.

A Key Artifact Paths

Artifact	Path
Current best public checkpoint	models/student_gru_hard_cases_radius1_seed601000.pt
Original best checkpoint	imitation_runs/dagger_gru_hard_cases_radius1_seed601000/checkpoints/student_hard_seed.pt
Best model note	models/CURRENT_BEST_STUDENT.md
Current best eval config	configs/imitation/eval_current_best_student.json
Balanced GRU config	configs/imitation/dagger_gru_balanced.json
GRU model source	src/imitation/models.py
Policy adapter source	src/imitation/policies.py
Evaluation helper	src/eval_student_checkpoints.py
Hard-case extractor	src/extract_hard_eval_seeds.py
Final training summary	imitation_runs/dagger_gru_hard_cases_radius1_seed601000/summaries/hard_seed_dataset_train_summary.json
Final 1000-episode eval	imitation_data/eval/dagger_gru_hard_cases_radius1_seed601000_1000eps/student_hard_seed_eval_1000eps.json
Final hard-case report	imitation_data/eval/dagger_gru_hard_cases_radius1_seed601000_1000eps/student_hard_seed_hard_cases.json

B Report Package Contents

This report folder contains:

- `main.tex`: LaTeX source.
- `Quanlong_Zhao_999014947_Final_Report.pdf`: compiled PDF.
- `figures/current_student_simulation.png`: simulation screenshot.
- `figures/training_pipeline.pdf`: generated pipeline figure.
- `figures/success_rate_ladder.pdf`: generated result ladder.
- `build_figures.py`: script used to regenerate the vector figures.